Katholieke
Universiteit
Leuven

**Department of
Computer Science**

A Reverse-Hoops-Playing Robot Arm

Robotics (B-KUL-H02A4A)

Enmin Lin (r0927480)

Academic year 2025–2026

Reverse-Hoops-Playing Robot Arm

System Thesis and Experimental Setup

This task admits a natural decomposition into estimation, feasibility-aware candidate selection, and constrained UR10 motion planning. The dominant design choice is the feasibility-aware selector. A better estimator alone does not solve the task, and a simple geometric catch point does not give a robot-executable catch. The experiments test this claim by holding the Kalman estimator and NLP controller fixed while changing the selection rule.

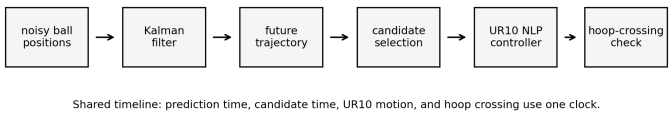

Figure 1: System pipeline used in the benchmark. The same ball prediction, candidate pool, NLP controller, and hoop-crossing metric support all strategy comparisons.

Table 1: Constants and modelling choices used by the estimator, controller, and success metric.

Item	Value	Quantitative role
Ball and hoop radii	0.12, 0.15 m	$r_h - r_b = 30\,\text{mm}$ radial tolerance
Measurement noise	1 mm Gaussian position noise	$R = \sigma^2 I_3$ in the Kalman update
Robot base	Fixed	Removes mobile-base planning from the report
Acceleration limit	$2\,\text{rad/s}^2$	$\lvert\ddot{q}_{k,j}\rvert \le 2$ for every knot and joint
Hoop orientation	Top side never faces ground	TCP z-axis satisfies $R_{zz}(q_k) \ge 0$
Collision evidence	Table clearance and self-sphere proxy	Conservative check, not exact mesh collision

The controller plans once after candidate selection and then replays the plan open-loop. It does not run MPC inside the simulation loop. The benchmark therefore separates planner quality from execution quality: terminal TCP error measures the NLP fit to the predicted target, while radial crossing error measures the final open-loop result against the true simulated ball trajectory. The simulator includes a bounce model, but all reported catches occur before bounce. A post-bounce catch would require a switching model or an interacting multiple-model filter.

Trajectory Estimation under Position Noise (addresses Task 1)

The ball state uses position and velocity,

$$x = \begin{bmatrix} p_x & p_y & p_z & v_x & v_y & v_z \end{bmatrix}^\top .$$

The process model integrates constant horizontal velocity and known gravity in z. The measurement model observes position only. With a Gaussian prior, linear dynamics, and linear Gaussian observations, the Kalman filter gives the Bayes-optimal MMSE update and an analytic covariance. An EKF collapses to the same linear update here, and a particle filter adds sampling noise without adding modelling power.

The filter initializes from the first noisy position measurement with zero velocity and a broad velocity covariance. The robot joint state remains perfectly observable in the simulator, so the estimator only tracks the ball. Across 50 seeds at the project noise level, raw measurement RMSE was 1.74 mm, filtered RMSE was 0.90 mm, and 0.5 s future-prediction RMSE was 3.51 mm. The 95th percentile future-prediction RMSE was 6.35 mm, well below the 30 mm hoop clearance.

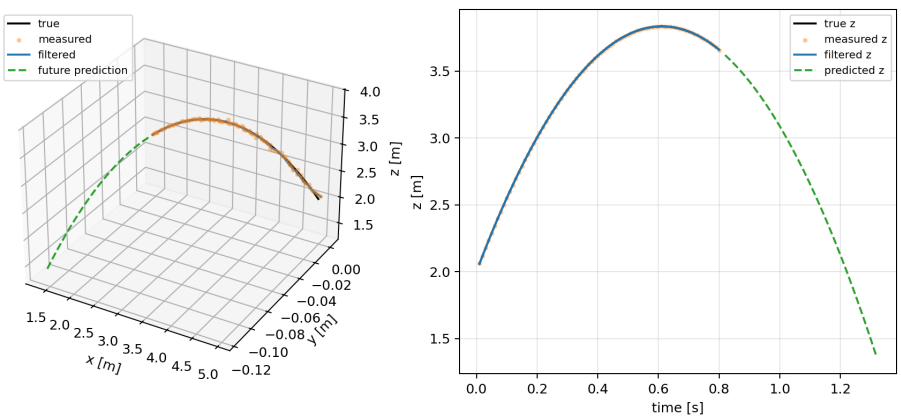

Figure 2: Kalman-filter prediction for one seed. The 50-seed validation keeps the 0.5 s prediction error below 6.35 mm at the 95th percentile, far inside the 30 mm clearance.

A noise sweep supports the same conclusion. When measurement noise increases from 1 to 10 mm, mean 0.5 s prediction RMSE increases from 3.67 to 10.34 mm, and the maximum observed RMSE remains 20.01 mm. A downstream Task 4 sweep over 20 seeds keeps time-first success at 95% for 1, 3, 5, and 10 mm noise. The estimator is therefore not the main bottleneck under the stated 1 mm setting.

The covariance output does not materially change selection in this benchmark. The propagation covariance is conservative under the chosen acceleration-process-noise model, while the empirical 0.5 s prediction error stays below the hoop clearance. A candidate-rejection ablation using the covariance selected the same catches as time-first feasible across all 50 seeds. The final selector keeps covariance as a diagnostic for future closed-loop variants, but it does not use a tuned covariance threshold in the open-loop benchmark.

Geometric Candidate Selection Baselines (addresses Task 2)

Task 2 asks for a simple interception point. The geometric baseline samples the predicted trajectory and rejects points that occur too soon, sit below $z = 0.35$ m, or lie too far from the current hoop center. It then chooses the earliest remaining point. For the validation seed, this rule selected $t = 1.36$ s and position $[1.224, -0.119, 1.074]$ m.

This rule answers where the ball could be caught in Cartesian space. It does not answer whether the UR10 can put the hoop there with the required pose and acceleration. Across 50 seeds, the simple geometric rule attempted an early catch at 1.295 ± 0.012 s, but it produced 0 successful catches under the full NLP and hoop-crossing check.

A second baseline removes prediction completely. It uses a damped Jacobian pseudo-inverse Cartesian velocity controller,

$$\dot{q} = J(q)^\top \left(J(q)J(q)^\top + \lambda^2 I \right)^{-1} K(p_{\text{ball,meas}} - p_{\text{tcp}}),$$

with proportional gain K, damping λ, and velocity and acceleration saturation. This reactive controller also gives 0% success. Its closest TCP-to-ball distance was 0.176 ± 0.058 m, and even the best seed stayed 59.4 mm away. The controller captures only the geometric closest-approach point of the flight path; it cannot accelerate the hoop fast enough to create a time-aligned catch. Prediction is therefore necessary, and a prediction-only geometric catch point remains insufficient.

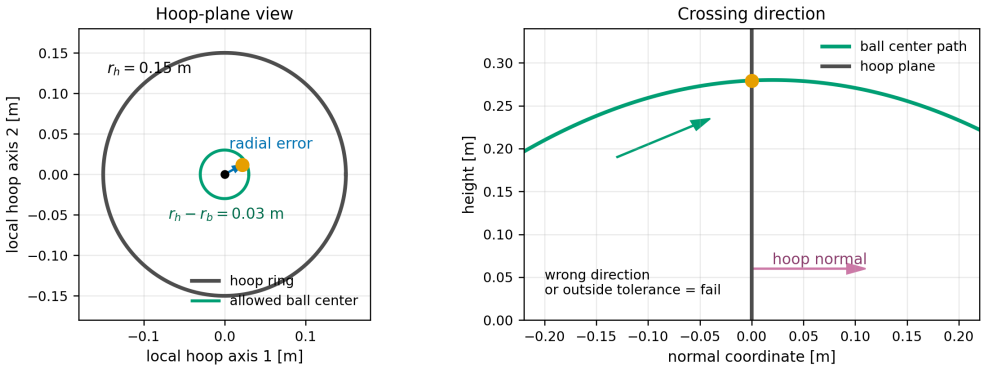

Figure 3: Success metric used in Task 3 and Task 4. The ball center must cross the hoop plane from the open side inside $r_h - r_b = 30$ mm; this is stricter than merely ignoring collision impulses.

Constrained Trajectory Optimization for the UR10 (addresses Task 3)

The controller solves a finite-horizon nonlinear program over joint position, velocity, and acceleration,

$$\{q_k, \dot{q}_k\}_{k=0}^{N}, \quad \{\ddot{q}_k\}_{k=0}^{N-1}.$$

It uses the second-order discrete dynamics

$$q_{k+1} = q_k + \dot{q}_k \Delta t + \tfrac{1}{2}\ddot{q}_k \Delta t^2, \qquad \dot{q}_{k+1} = \dot{q}_k + \ddot{q}_k \Delta t.$$

The optimization problem is

$$\begin{aligned}
\min_{q,\dot{q},\ddot{q}} \; & w_p \|p_{\text{tcp}}(q_N) - p^\star\|_2^2 + \sum_{k=0}^{N-1} w_u \|\ddot{q}_k\|_2^2 + \sum_{k=1}^{N} w_v \|\dot{q}_k\|_2^2 \\
& + w_o (1 - R_{zz}(q_N))^2 + w_n \left(1 - (n_{\text{hoop}}(q_N)^\top \hat{v}_{\text{ball}})^2\right) \\
\text{s.t. } & q_0 = q_{\text{init}}, \quad \dot{q}_0 = \dot{q}_{\text{init}}, \\
& q_{\min} \le q_k \le q_{\max}, \quad -\dot{q}_{\max} \le \dot{q}_k \le \dot{q}_{\max}, \quad -\ddot{q}_{\max} \le \ddot{q}_k \le \ddot{q}_{\max}, \\
& z_{\text{tcp}}(q_k) \ge h_{\text{table}} + m_{\text{tcp}}, \quad R_{zz}(q_k) \ge 0, \qquad \forall k.
\end{aligned}$$

The validation uses $N = 28$, $\Delta t = 0.05$ s, $w_p = 1500$, $w_u = 2 \times 10^{-2}$, $w_v = 10^{-3}$, $w_o = 10$, and $w_n = 5$. The hoop mesh is thin along the x-axis of `tool_link`. The fixed TCP joint rotates this physical hoop normal into the local z-axis of the `tcp` frame. The implementation uses TCP z as n_{hoop}, and the same TCP z-axis defines the top-side path constraint $R_{zz}(q_k) \ge 0$. The

open side of the hoop points along $+n_{\text{hoop}}$, so the crossing check requires the ball to enter through that side before applying the radial tolerance. Pinocchio supplies URDF forward kinematics. CasADi builds the symbolic NLP, and IPOPT solves it.

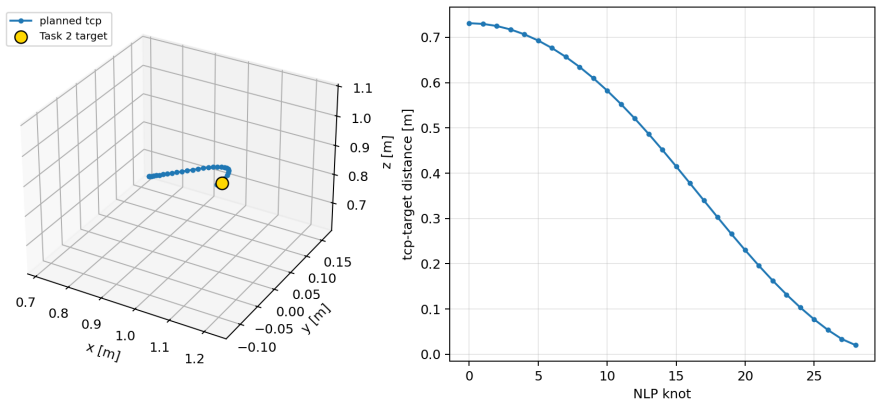

Figure 4: Task 3 finite-horizon NLP result. The hoop center reaches 20.1 mm terminal TCP error while the path constraints keep the motion executable.

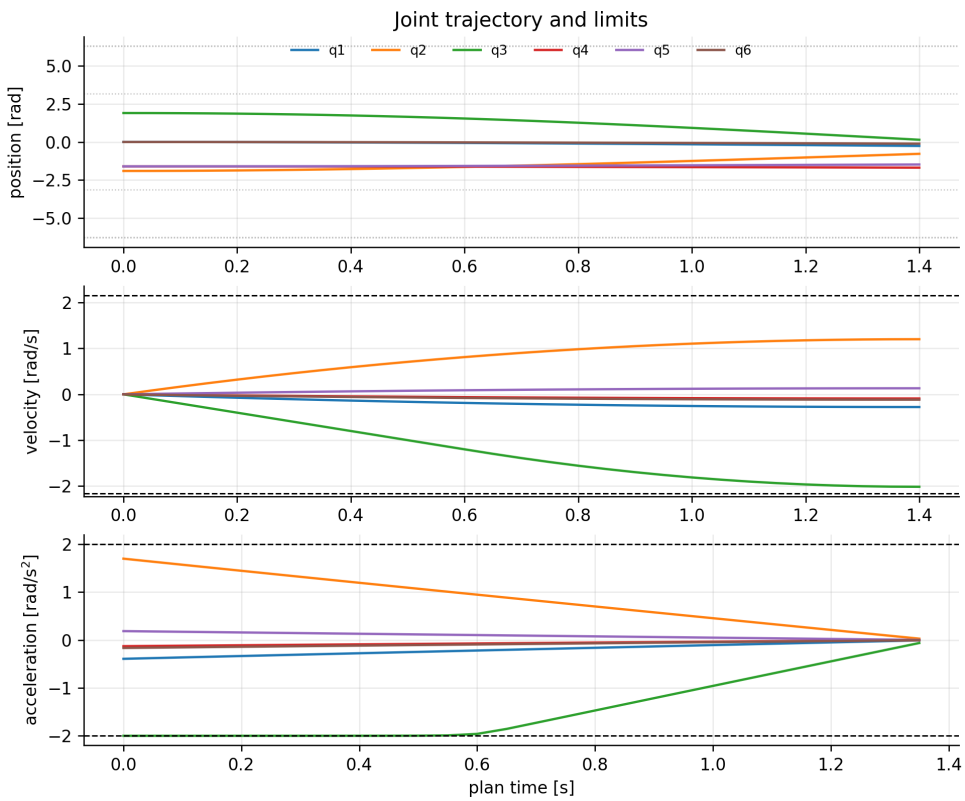

Figure 5: Joint profiles for the same plan. All accelerations approach but do not exceed $\pm 2\,\text{rad/s}^2$, and joint 2 uses the largest velocity margin at 64% of its limit.

For the Task 2 target, the NLP reduced the hoop-target distance from 0.731 m to 0.020 m. It solved in 0.071 s and 15 IPOPT iterations on the validation run. The maximum acceleration was $1.9996\,\text{rad/s}^2$, the maximum velocity ratio was 0.640, and the minimum R_{zz} value was 0.732. The minimum TCP-table clearance was 0.622 m, the minimum frame-table clearance was 0.115 m, and the minimum self-sphere clearance was 0.163 m. Table clearance and hoop orientation enter the NLP as hard path constraints. Self-collision remains a post hoc sphere-based proxy. An ablation over $w_o \in \{0, 1, 10, 50\}$ kept time-first benchmark success at 98%. This shows that the hard $R_{zz} \geq 0$ path constraint and the pose-aware normal term set the final geometry; the terminal R_{zz} cost mainly regularizes the pose.

Feasibility-Aware Candidate Selection (addresses Task 4)

Task 4 asks for higher success rate and shorter interception time. The result depends on the comparison domain. The simple geometric rule attempts an earlier catch, but it has 0% success, so its attempted time is not a successful-interception baseline. The fair time comparison uses successful feasible policies evaluated on the same candidate pool.

The selector forms a shared candidate pool from the predicted trajectory, solves the Task 3 NLP for each shortlisted candidate, and accepts only candidates that pass three checks: NLP convergence, hoop-plane crossing by the true ball trajectory, and zero safety penalty. All 396 evaluated candidate NLPs converged, but only 164 candidates passed the hoop-crossing test.

Each seed therefore produced 7.9 evaluated candidates and 3.3 feasible catches on average. The feasibility filter rejects about 59% of geometrically plausible candidates. This split identifies the bottleneck: the robot can solve the motion plans, but many candidate times do not place the hoop in a crossing-compatible pose. Compared with the simple geometric attempt at 1.295 s, the time-first feasible selector spends about 92 ms and raises success from 0% to 98%.

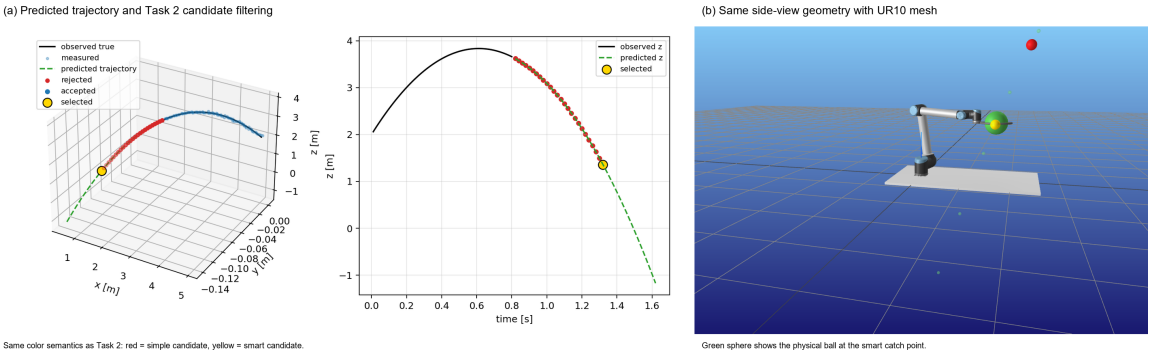

Figure 6: Task 2 and Task 4 in a shared side view. The simple candidate is earlier but infeasible; the selected feasible candidate aligns the UR10 hoop with a true crossing.

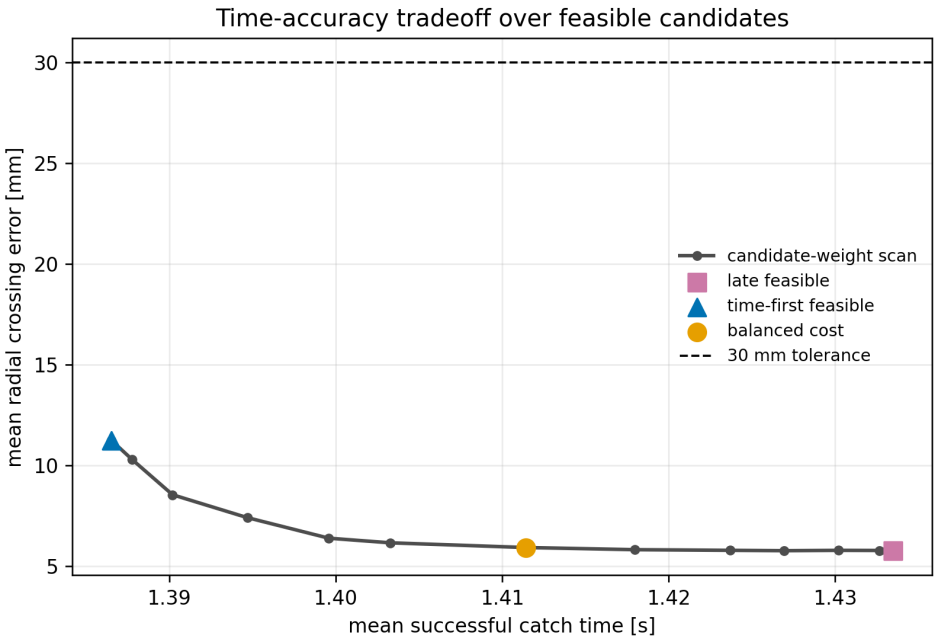

Figure 7: Empirical candidate-weight scan on the time-accuracy plane. Time-first feasible gives the early-time end, balanced cost gives a tunable midpoint, and late feasible gives the accurate but slower end.

Table 2: Final 50-seed benchmark. Successful time, errors, effort, solve time, and iterations use successful cases. The simple time is an attempted time only.

Strategy	Success	Time [s]	TCP [mm]	Radial [mm]	Max $\ddot{q}$	IPOPT
Reactive no prediction	0%	–	–	–	–	–
Simple geometric	0%	1.295*	–	–	–	–
Late feasible	98%	1.433 ± 0.009	1.87	5.75	1.65	10.4 it.
Time-first feasible	98%	1.387 ± 0.018	13.18	11.24	2.00	13.5 it.
Balanced cost	98%	1.411 ± 0.015	3.22	5.91	1.81	12.3 it.

*Attempted catch time, not successful interception time.

The time-first feasible policy gives the strongest answer to the Task 4 wording. It raises full-system success from 0% to 98% and catches 0.025 s earlier than balanced cost and 0.047 s earlier than late feasible. The latter gap is about one control tick. Late feasible gives the best accuracy, while balanced cost uses a time weight of 15.0 in the selector cost and lands between the two extremes. The submitted Task 4 answer is time-first because the wording explicitly asks for shorter interception time. If the objective weighted hoop-crossing accuracy more heavily, balanced cost or late feasible would be the stronger engineering choice. Figure 7 makes this trade visible.

Time-first selects the earliest feasible candidate, so it runs with the tightest motion budget. The NLP saturates the acceleration bound and has less freedom to refine pose. Balanced and late-feasible candidates occur later, where the NLP has slack to align the hoop normal with the ball direction. This explains the lower TCP and radial errors in those rows. Late feasible reaches below 2 mm TCP error, while radial error remains near 6 mm, because the true ball crosses the hoop plane slightly away from the predicted target after measurement-noise-driven prediction error and candidate-time discretization.

The open-loop model-execution gap remains small. For successful time-first catches, the NLP terminal error to the predicted target averaged 13.18 mm, the predicted hoop-crossing radial error averaged 9.36 mm, and the true-ball radial error averaged 11.24 mm. The difference of means is 1.87 mm, while the per-seed mean absolute predicted-to-true radial gap is 4.41 mm. This separates solver quality from execution quality without hiding the open-loop assumption.

Failure Analysis, Course Connection, and Reproducibility

The only time-first failure is seed 11. It has high positive lateral velocity v_y, close to the 95th percentile of the 50-seed distribution. The corrected TCP z-axis hoop normal reduced seed 11's best radial error from 145.2 to 51.5 mm, but this still exceeds the 30 mm tolerance. The same correction recovered seed 41, which now crosses at 24.7 mm radial error. A mitigation run with 16 candidates and a 1.35 m workspace radius did not recover seed 11. The remaining failure is not estimator divergence or NLP non-convergence; it is a residual lateral-trajectory geometry case near the edge of the evaluated catch set. Recovering it would require a longer prediction horizon or a different candidate sampler.

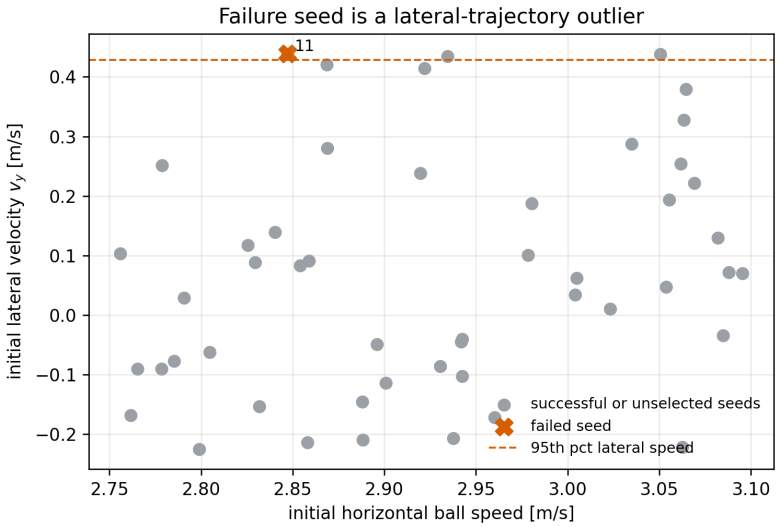

Figure 8: Failure mechanism for seed 11. The red marker is the failed seed and the grey markers are the other seeds; seed 11 sits near the 95th percentile of lateral velocity and fails by geometry, not estimator divergence or NLP failure.

The method maps directly to the course material. The hard inequalities follow constraint-based robot programming from C4. The finite horizon over q, $\dot{q}$, and $\ddot{q}$ is a direct-transcription optimal control problem from C5. The objective is a soft multi-objective formulation rather than a strict stack of tasks from C3. A hard hierarchical stack was not used because joint limits, acceleration limits, hoop orientation, and table clearance have equal physical priority once they become hard constraints. A single-step resolved acceleration controller can track a Cartesian target, but it cannot enforce the path constraints over the full prediction window. A minimum-jerk trajectory handles smoothness, but it does not handle the hoop crossing and joint-limit constraints by itself.

The experiments ran with Python 3.13.13, NumPy 2.4.3, CasADi 3.6.7, Pinocchio 3.4.0, and IPOPT through CasADi. The main benchmark uses seeds 0–49. The noise sweep uses seeds 0–49 for prediction error and seeds 0–19 for downstream success. The main reproduction commands are:

`python scripts/benchmark_high_score_strategies.py` and `python scripts/render_high_score_figures.py`.

Conclusion

The final system supports the thesis. Prediction is necessary, because the reactive controller never reaches the ball within the tolerance. Prediction alone is insufficient, because the simple geometric selector also gives 0% success. The decisive step is feasibility-aware candidate selection: it keeps the estimator and controller fixed, filters candidates through the UR10 NLP and the true hoop-crossing geometry, and raises success to 98%. Time-first feasible gives the shortest successful catch among the evaluated policies, while the balanced selector shows the measurable cost of buying a few millimetres of radial accuracy.

References

1. Robotics course project description, *A Reverse-Hoops-Playing Robot Arm*, KU Leuven, 2025–2026.
2. KU Leuven Robotics lecture C6, *Probabilistic Robotics*: Bayes filters, Kalman filters, and tracking process models.
3. KU Leuven Robotics lecture C3, *Task-Space Control*: resolved motion control, resolved acceleration control, damped pseudo-inverse, and stack of tasks.
4. KU Leuven Robotics lecture C4, *Constraint-Based Robot Programming*: hard and soft constraints, second-order constraints, and instantaneous optimization.
5. KU Leuven Robotics lecture C5, *Motion Planning*: configuration space, free space, signed distance, sampling, and optimal control.
6. R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, 82(1), 35–45, 1960. DOI: 10.1115/1.3662552.
7. A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical Programming*, 106, 25–57, 2006. DOI: 10.1007/s10107-004-0559-y.
8. J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi: a software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, 11, 1–36, 2019. DOI: 10.1007/s12532-018-0139-4.
9. J. Carpentier et al., "The Pinocchio C++ library: a fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives," IEEE/SICE International Symposium on System Integration, 2019.
10. R. Featherstone, *Rigid Body Dynamics Algorithms*. Springer, 2008. DOI: 10.1007/978-1-4899-7560-7.